

# Adversarial Search

## Chapter 6

# Outline

- Introduction
- Optimal decisions
- $\alpha$ - $\beta$  pruning
- Imperfect, real-time decisions
- AlphaGo
- Non-deterministic games

# Games vs. search problems

- We use search to solve games between opponents: **adversarial search**
  - 确定性的、两个游戏者轮流行动的、完整信息的零和游戏
- "Unpredictable" opponent → specifying a move for every possible opponent reply
- 
- Time limits → unlikely to find goal, must approximate
-

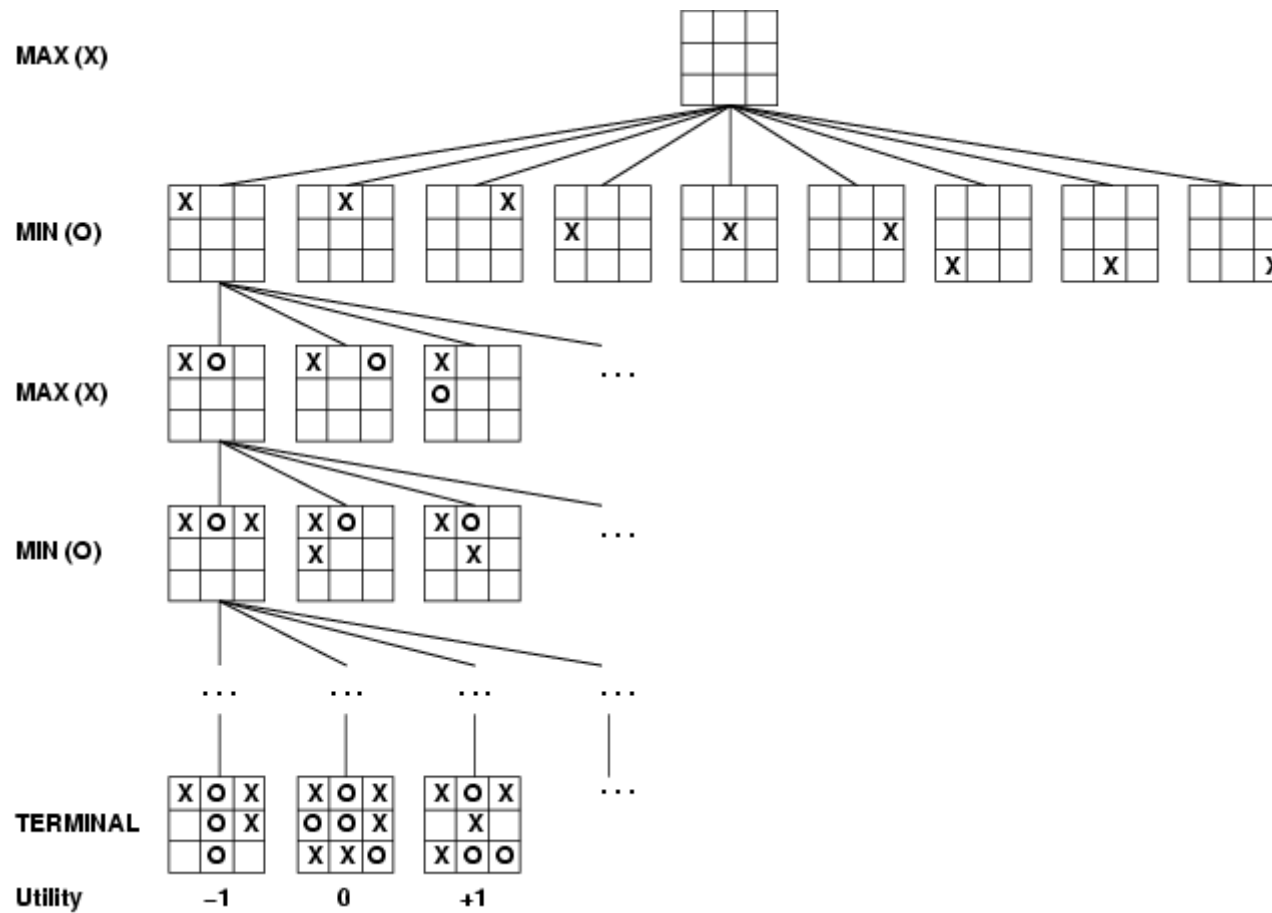
# Type of games

|                       | deterministic                   | chance                                 |
|-----------------------|---------------------------------|----------------------------------------|
| perfect information   | chess, checkers,<br>go, othello | backgammon<br>monopoly                 |
| imperfect information | battleships,<br>blind tictactoe | bridge, poker, scrabble<br>nuclear war |

# 问题描述

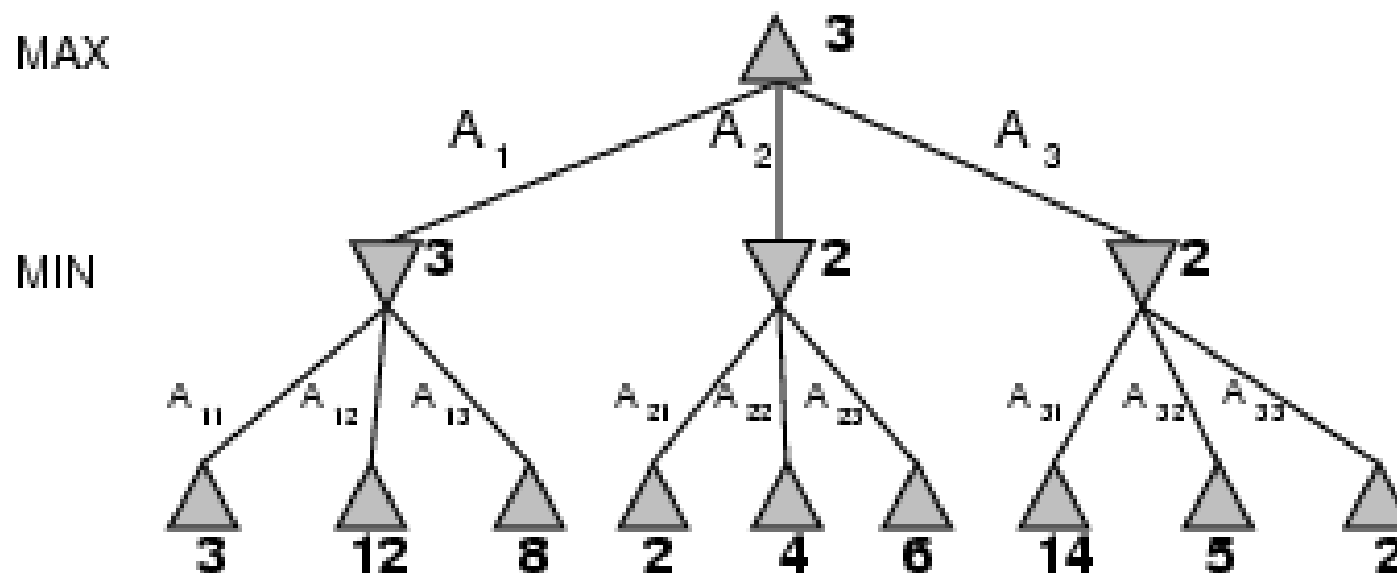
- 两名游戏者**MAX**和**MIN**，**MAX**先行，然后两人轮流走招，直到游戏结束。
  - 初始状态，棋盘局面
  - 后继函数，返回(move, state)
  - 终止测试，是否终局
  - 效用函数：对终止状态给出一个确定的数值
    - Utility based agent
- **MAX**要使其效用最大，使对手的效用最小

# Game tree (2-player, deterministic, turns)



# Optimal decisions

- 最优决策是导致终局取胜的一系列招数，它保证在对手不犯错误时，能够导致至少不比任何其它策略差的结果。



一颗两层的博弈树

# Minimax

- Perfect play for deterministic games
- 
- Idea: choose move to position with highest **minimax value**  
= best achievable payoff against best play

$$MINIMAX - VALUE(n) =$$

$$\begin{cases} UTILITY(n), \text{ if } n \text{ is a terminal node} \\ \max_{s \in Successors(n)} MINIMAX - VALUE(s), \text{ if } n \text{ is a MAX node} \\ \min_{s \in Successors(n)} MINIMAX - VALUE(s), \text{ if } n \text{ is a MIN node} \end{cases}$$



# Minimax algorithm

**function** MINIMAX-DECISION(*state*) *returns an action*

$v \leftarrow \text{MAX-VALUE}(\textit{state})$

**return** the *action* in SUCCESSORS(*state*) with value *v*

---

**function** MAX-VALUE(*state*) *returns a utility value*

**if** TERMINAL-TEST(*state*) **then return** UTILITY(*state*)

$v \leftarrow -\infty$

**for** *a, s* in SUCCESSORS(*state*) **do**

$v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s))$

**return** *v*

---

**function** MIN-VALUE(*state*) *returns a utility value*

**if** TERMINAL-TEST(*state*) **then return** UTILITY(*state*)

$v \leftarrow \infty$

**for** *a, s* in SUCCESSORS(*state*) **do**

$v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(s))$

**return** *v*

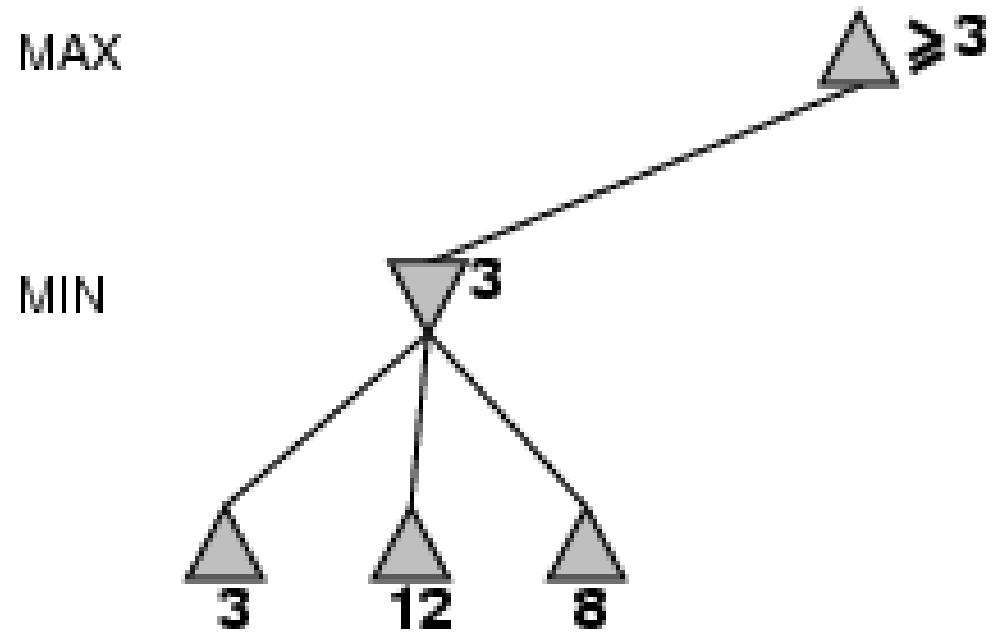
# Properties of minimax

- Complete? Yes (only if tree is finite)
- 
- Optimal? Yes (against an optimal opponent. Otherwise?)
- 
- Time complexity?  $O(b^m)$
- 
- Space complexity?  $O(bm)$  (depth-first exploration)
- 
- For chess,  $b \approx 35$ ,  $m \approx 100$   
→ exact solution completely infeasible
- 
- But do we need to explore every path?

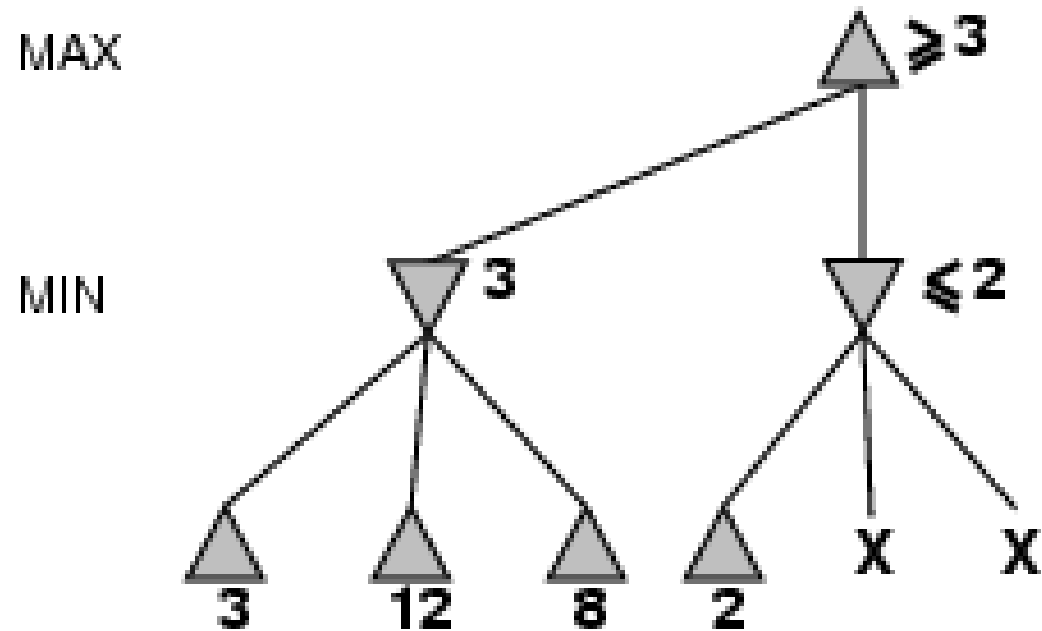
# $\alpha$ - $\beta$ 剪枝

- 剪枝：剪掉那些不影响最后决策的分支
- 一般原则：考虑在树中某处的节点 $n$ ，如果游戏者在 $n$ 的父节点或上层的任何节点有一个更好的选择 $m$ ，那么在实际的游戏中就永远不会到达 $n$ ，即 $n$ 可被剪掉。

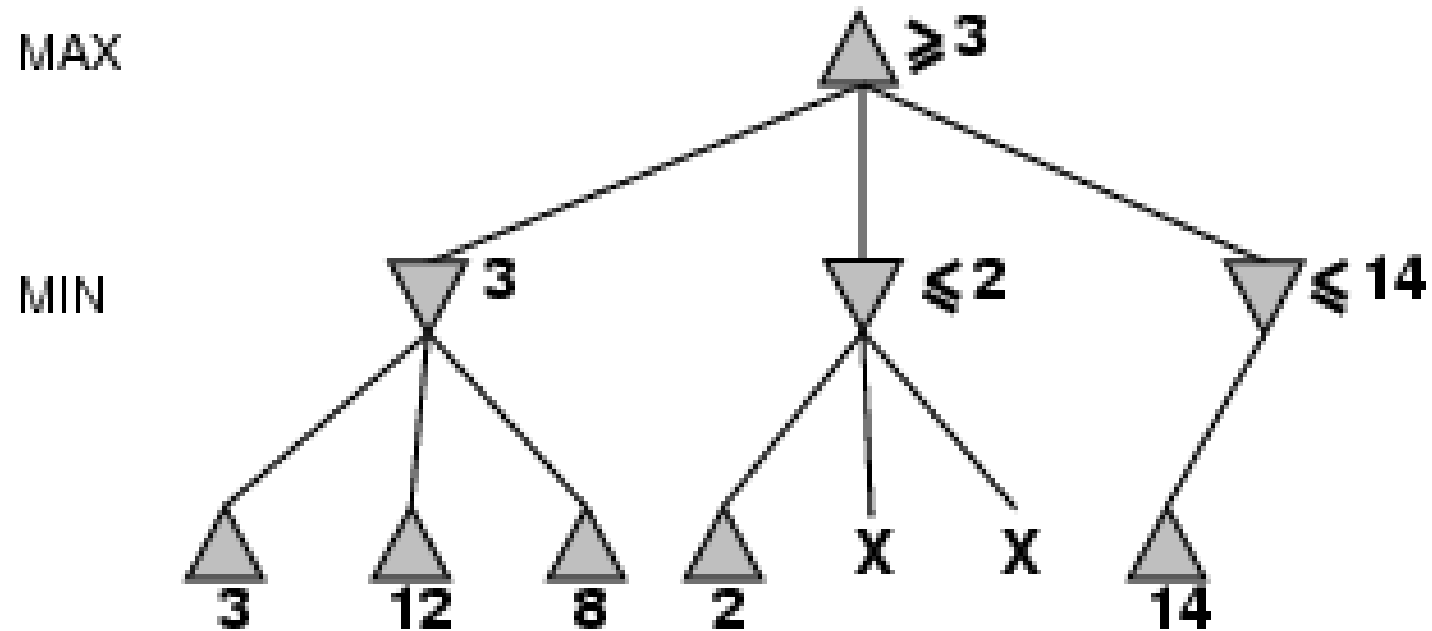
# $\alpha$ - $\beta$ pruning example



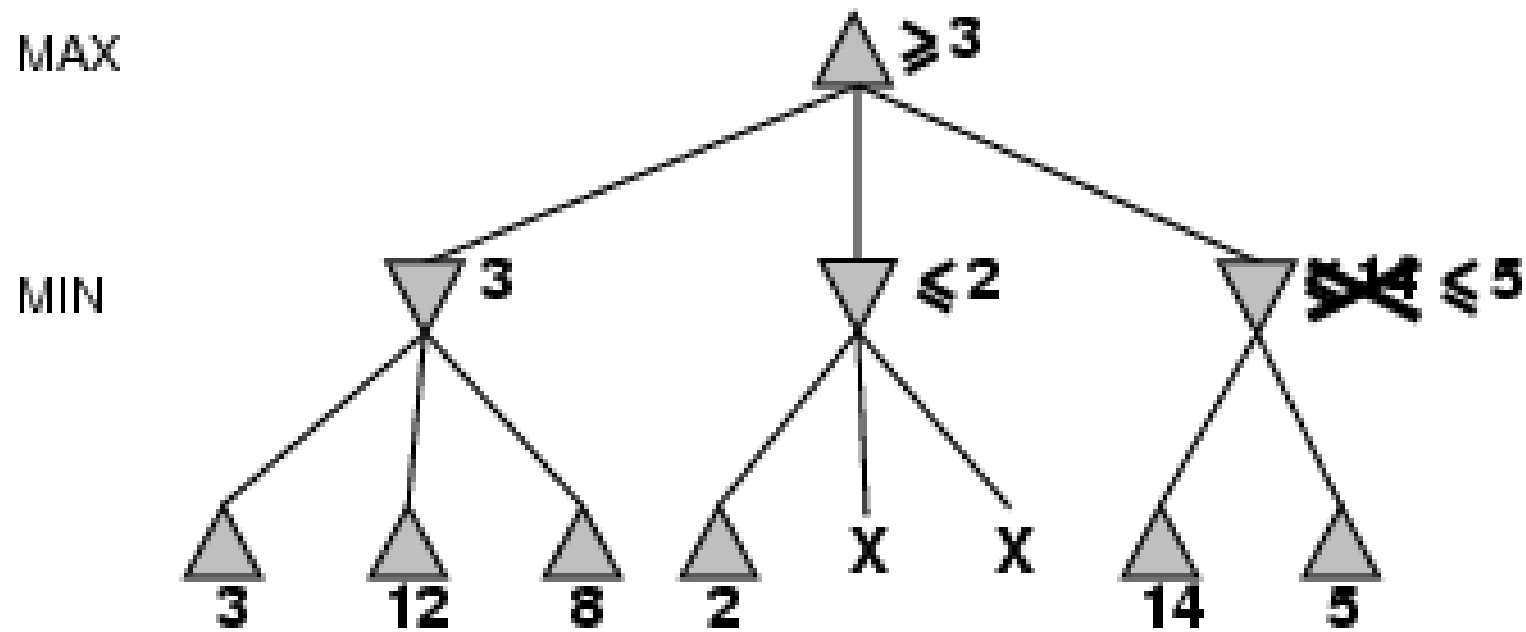
# $\alpha$ - $\beta$ pruning example



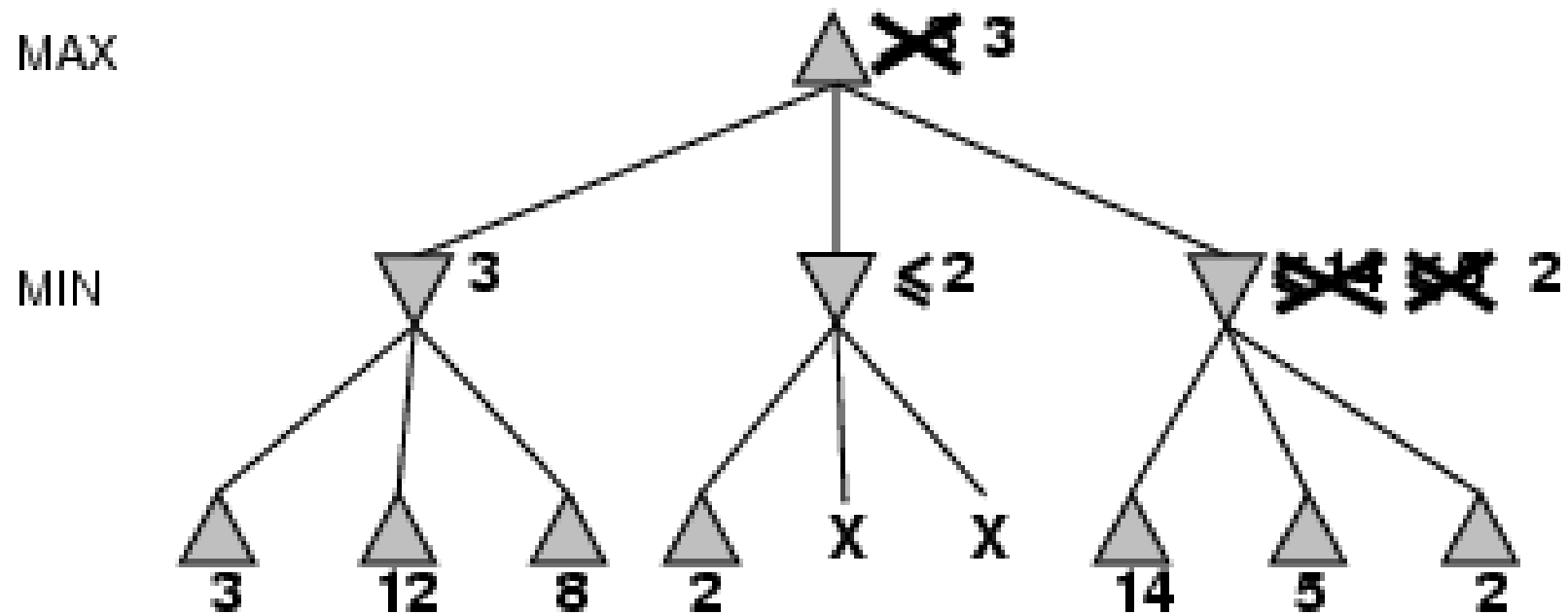
# $\alpha$ - $\beta$ pruning example



# $\alpha$ - $\beta$ pruning example



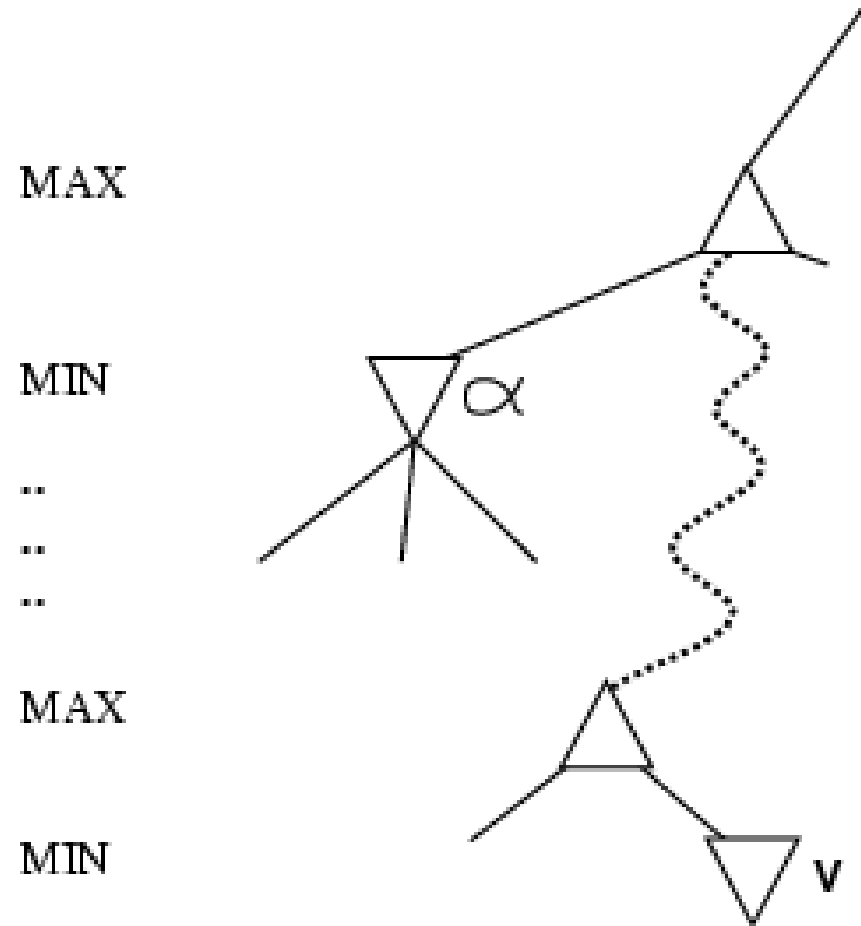
# $\alpha$ - $\beta$ pruning example





# Why is it called $\alpha$ - $\beta$ ?

- $\alpha$  is the value of the best (i.e., highest-value) choice found so far at any choice point along the path for *max*
- If  $v$  is worse than  $\alpha$ , *max* will avoid it
- $\rightarrow$  prune that branch
- Define  $\beta$  similarly for *min*
- 



# Properties of $\alpha$ - $\beta$

- $\alpha$  = 到目前为止在路径上的任意选择点发现的MAX的最佳选择
- $\beta$  = 到目前为止在路径上的任意选择点发现的MIN的最佳选择
- 当某个节点的值比目前MAX的 $\alpha$ 值或MIN的 $\beta$ 值更差时，裁掉该节点剩下的分支
- Pruning **does not** affect final result
- Good move ordering improves effectiveness of pruning
- 
- With "perfect ordering," time complexity =  $O(b^{m/2})$   
→ **doubles** depth of search

Unfortunately,  $35^{50}$  is still impossible!

# The $\alpha$ - $\beta$ algorithm

**function** ALPHA-BETA-SEARCH(*state*) *returns an action*

**inputs:** *state*, current state in game

$v \leftarrow \text{MAX-VALUE}(\text{state}, -\infty, +\infty)$

**return** the *action* in SUCCESSORS(*state*) with value *v*

---

**function** MAX-VALUE(*state*,  $\alpha$ ,  $\beta$ ) *returns a utility value*

**inputs:** *state*, current state in game

$\alpha$ , the value of the best alternative for MAX along the path to *state*

$\beta$ , the value of the best alternative for MIN along the path to *state*

**if** TERMINAL-TEST(*state*) **then return** UTILITY(*state*)

$v \leftarrow -\infty$

**for** *a, s* in SUCCESSORS(*state*) **do**

$v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s, \alpha, \beta))$

**if**  $v \geq \beta$  **then return** *v*

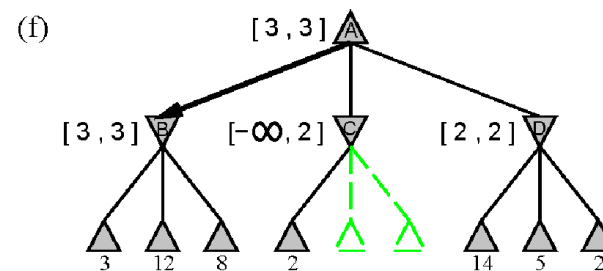
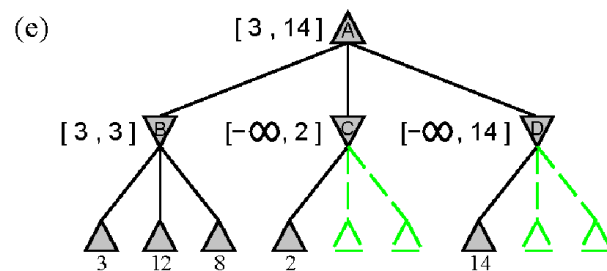
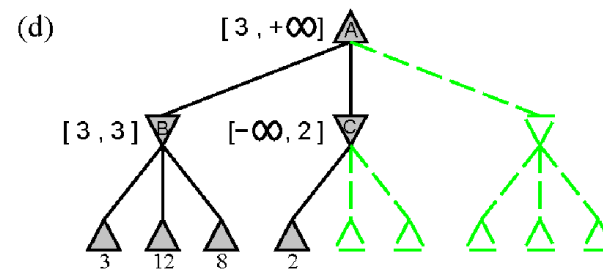
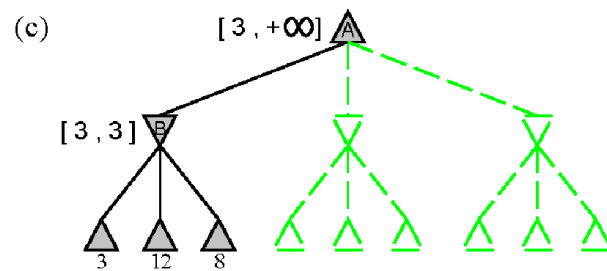
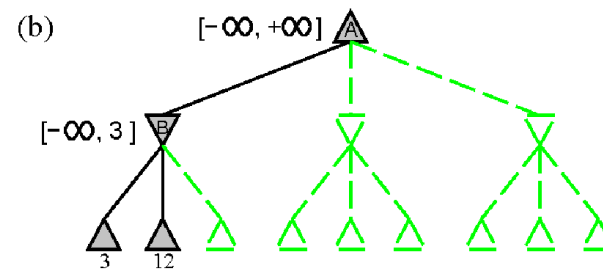
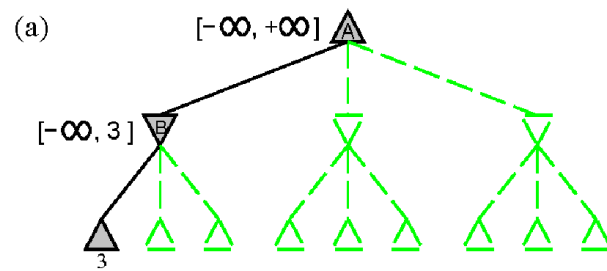
$\alpha \leftarrow \text{MAX}(\alpha, v)$

**return** *v*

# The $\alpha$ - $\beta$ algorithm

```
function MIN-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value
  inputs: state, current state in game
            $\alpha$ , the value of the best alternative for MAX along the path to state
            $\beta$ , the value of the best alternative for MIN along the path to state

  if TERMINAL-TEST(state) then return UTILITY(state)
   $v \leftarrow +\infty$ 
  for  $a, s$  in SUCCESSORS(state) do
     $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(s, \alpha, \beta))$ 
    if  $v \leq \alpha$  then return  $v$ 
     $\beta \leftarrow \text{MIN}(\beta, v)$ 
  return  $v$ 
```



# Resource limits

Suppose we have 100 secs, explore  $10^4$  nodes/sec

$\Rightarrow 10^6$  nodes per move  $\approx 35^{8/2}$

$\Rightarrow \alpha\text{-}\beta$  reaches depth 8  $\Rightarrow$  pretty good chess program

# Imperfect, real-time decisions

- $\alpha$ - $\beta$ 算法虽然减少了搜索空间，但对人玩的游戏无法搜索到终止状态。
- Standard approach:
- **cutoff test: (instead of terminal-test)**  
e.g., depth limit (perhaps add **quiescence search**)
  - **evaluation function (instead of Utility)**  
= estimated desirability of position

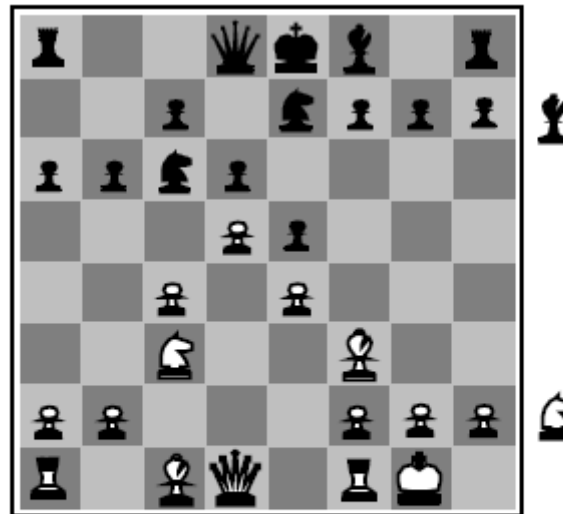
# Evaluation functions

- For chess, typically **linear** weighted sum of **features**

$$Eval(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$$

- e.g.,  $w_1 = 9$  with

$f_1(s) = (\text{number of white queens}) - (\text{number of black queens}), \text{ etc.}$



Black to move

White slightly better



# Cutting off search

*MinimaxCutoff* is identical to *MinimaxValue* except

1. *Terminal?* is replaced by *Cutoff?*
2. *Utility* is replaced by *Eval*

Does it work in practice?

$$b^m = 10^6, b=35 \rightarrow m=4$$

4-ply lookahead is a hopeless chess player!

- 4-ply  $\approx$  human novice
- 8-ply  $\approx$  typical PC, human master
- 12-ply  $\approx$  Deep Blue, Kasparov
-

# quiescence search

- 评价函数应该只用于那些静止的棋局（不会出现拐点），非静止的棋局可以进一步扩展直到静止的棋局，这种额外的搜索称为*静止搜索*。

# 避免重复状态

- 在游戏中重复状态的频繁出现：导致同样棋局的不同行棋序列的排列
  - 例如， $[a_1, b_1, a_2, b_2]$ 和 $[a_1, b_2, a_2, b_1]$ 都结束于同样的棋局，其中 $[b_1, b_2]$ 是 $[b_2, b_1]$ 的置换。
- 解决办法：第一次遇见某棋局时将对它的评价存储在哈希表中（该哈希表称为置换表）。

# Deterministic games in practice

- Checkers: Chinook ended 40-year-reign of human world champion Marion Tinsley in 1994. Used a precomputed endgame database defining perfect play for all positions involving 8 or fewer pieces on the board, a total of 444 billion positions.
- Chess: Deep Blue defeated human world champion Garry Kasparov in a six-game match in 1997. Deep Blue searches 200 million positions per second, uses very sophisticated evaluation, and undisclosed methods for extending some lines of search up to 40 ply.
- 
- Othello: human champions refuse to compete against computers, who are too good.
- 
- Go: AlphaGo defeated No.4 player [Lee Sedol](#) in March 2016. It is believed that no human player can compete with it.
- .
-

A photograph of Lee Sedol, a professional Go player, sitting at a Go board. He is wearing a dark suit and a light blue shirt. He has his hand on his forehead, looking down at the board with a focused and slightly stressed expression. The Go board is made of light-colored wood and is filled with black and white stones. In the background, another person's arm is visible, and a blue screen with some text is partially visible.

I heard Google DeepMind's AI is surprisingly strong and getting stronger, but I am confident that I can win, at least this time.

---

Lee Sedol

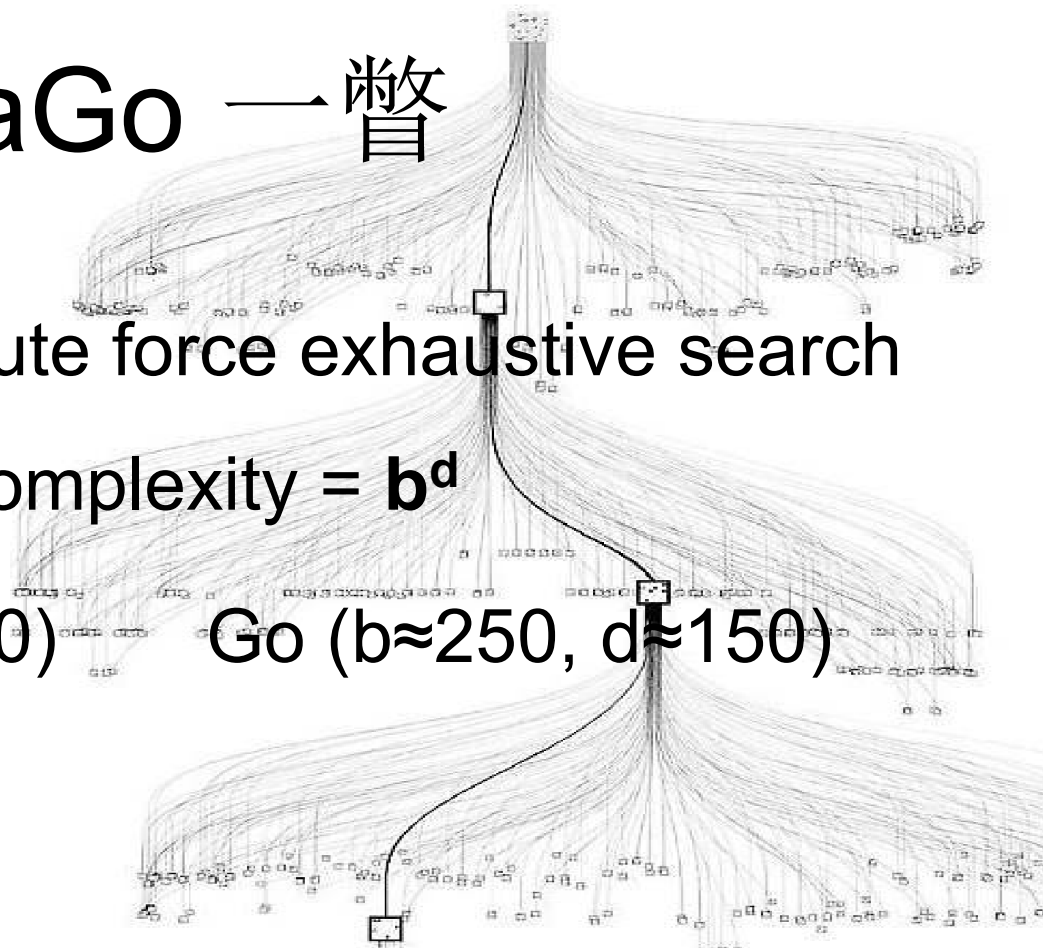
...even beating AlphaGo by 4:1 may allow the Google DeepMind team to claim its de facto victory and the defeat of him [Lee Sedol], or even humankind.

---

interview in JTBC  
Newsroom

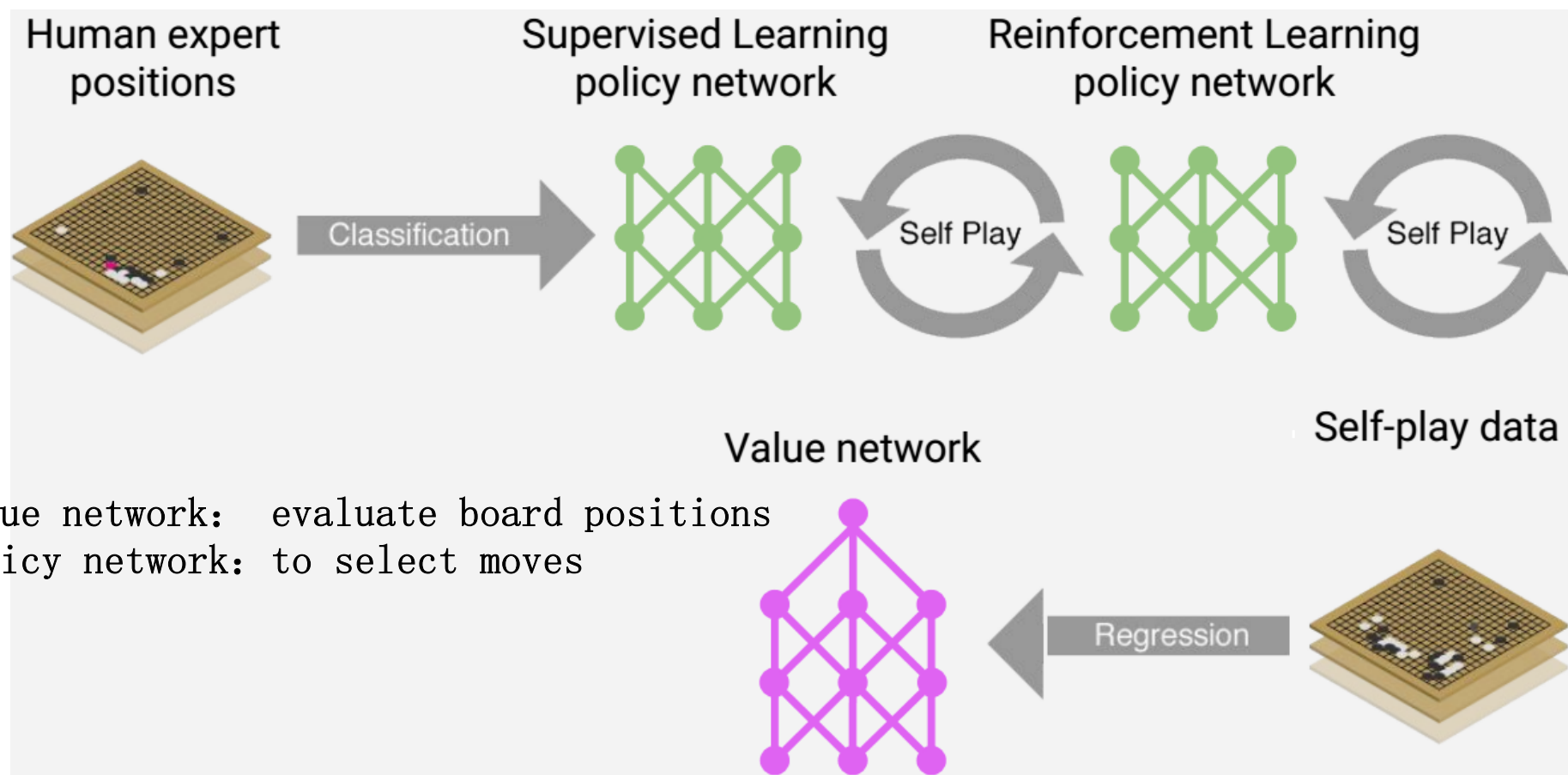
# AlphaGo 一瞥

- Impossible to use brute force exhaustive search in Go: Game tree complexity =  $b^d$ 
  - Chess ( $b \approx 35$ ,  $d \approx 80$ )      Go ( $b \approx 250$ ,  $d \approx 150$ )
- AlphaGo 关键词
  - 深度卷积神经网络
  - 策略 (Policy) 网络、价值 (Value) 网络
  - 蒙特卡罗树搜索 (MCTS)
  - 监督学习 (SL)、强化学习 (RL)

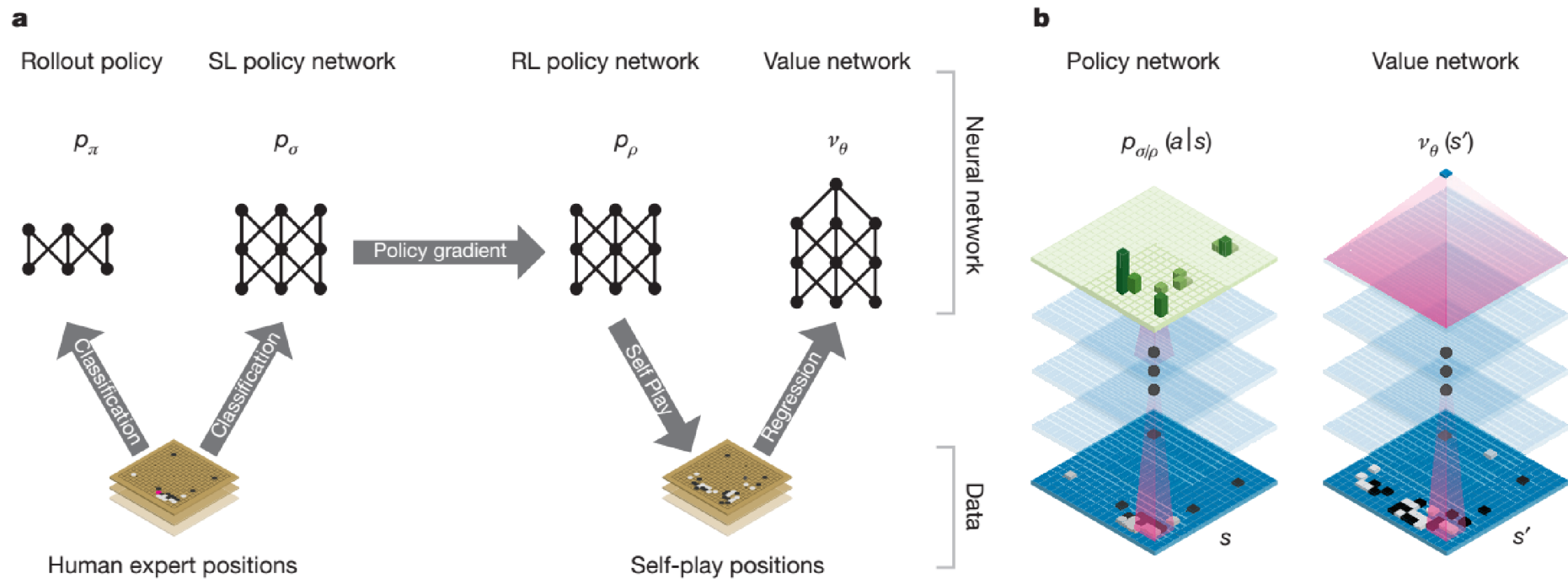


# AlphaGo 一瞥

- 训练流程:



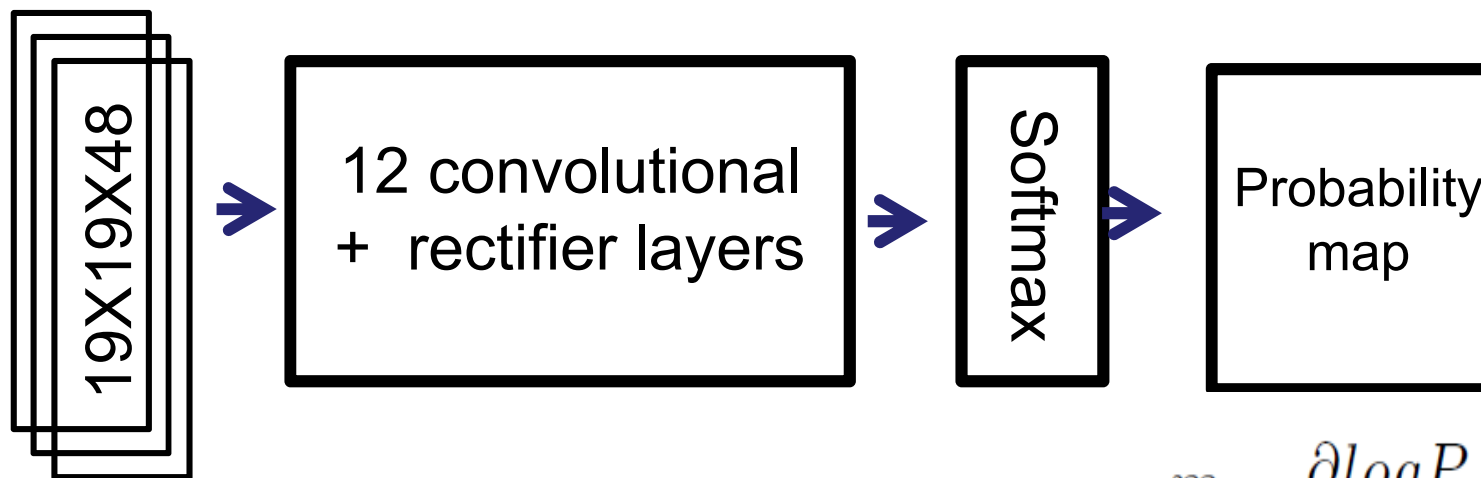
# AlphaGo 一瞥





# SL Policy Network : $P_{\sigma}$

- ~30 million (state, action) from KGS go server
- Goal: maximize the log likelihood  $\underset{\sigma}{argmax} (\log (P_{\sigma} (a|s)))$
- Input : 48 feature planes
- Output: action probability map



step size init  $\alpha = 0.003$  and  $m = 16$

$$\Delta\sigma = \frac{\alpha}{m} \sum_{k=1}^m \frac{\partial \log P_{\sigma}(a^k | s^k)}{\partial \sigma}$$

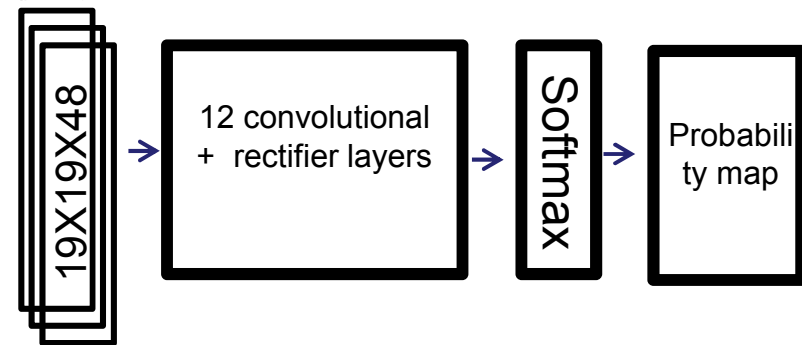
# Training the RL Policy Network $P_\rho$

Refined version of SL policy ( $P_6$ )

Initialize weights to 0=5

$\mu = \mu$  is an old version of  $\rho$

$J_\rho$  vs  $J_{\mu}$



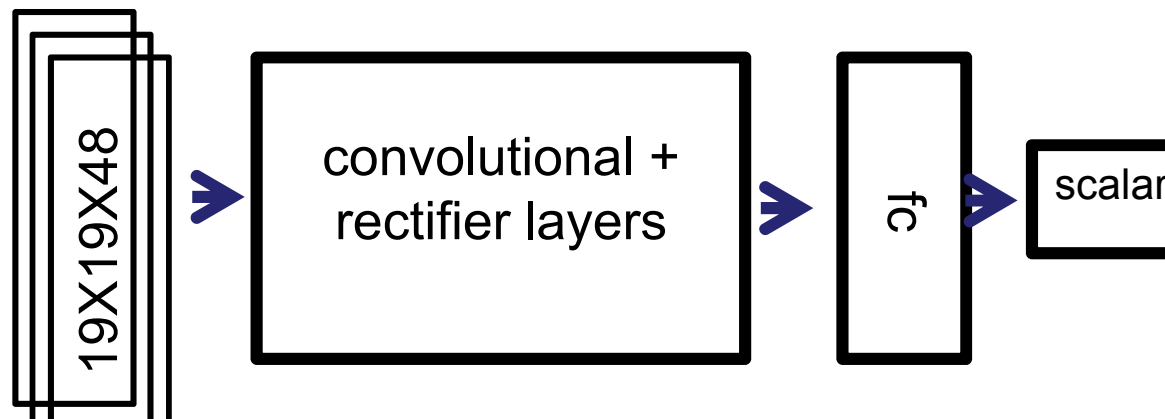
## SGA

$$s_1 \xrightarrow{a_1} s_2 \xrightarrow{a_2} \dots \xrightarrow{a_{T-1}} s_T \Rightarrow z_t = r(s_T) \in \{\pm 1\} \quad \Delta \rho \propto \sum_{t=1}^T \frac{\partial \log P_\rho(a_t | s_t)}{\partial \rho} z_t$$

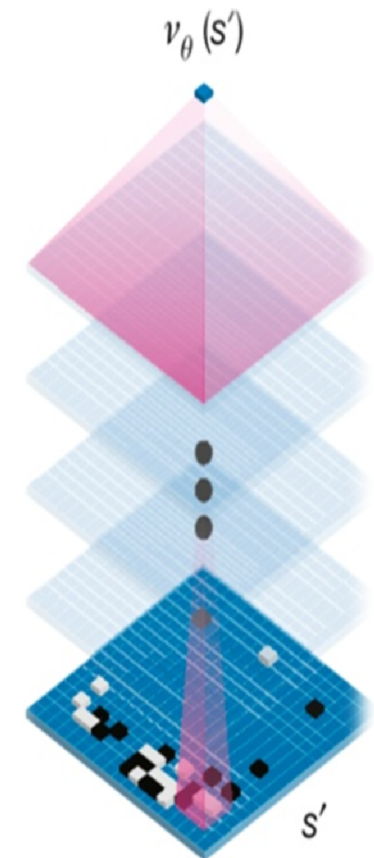
- Preventing overfitting
- RL policy Won more then 80% of the games against SL policy

# Training the Value Network $V_\theta$

- Position evaluation
- Approximating optimal value function  $v_\theta(s) \approx v^{p_\rho}(s) \approx v^*(s)$
- Input : state , output: probability to win
- Goal: minimize MSE
- Overfitting - position within games are strongly correlated

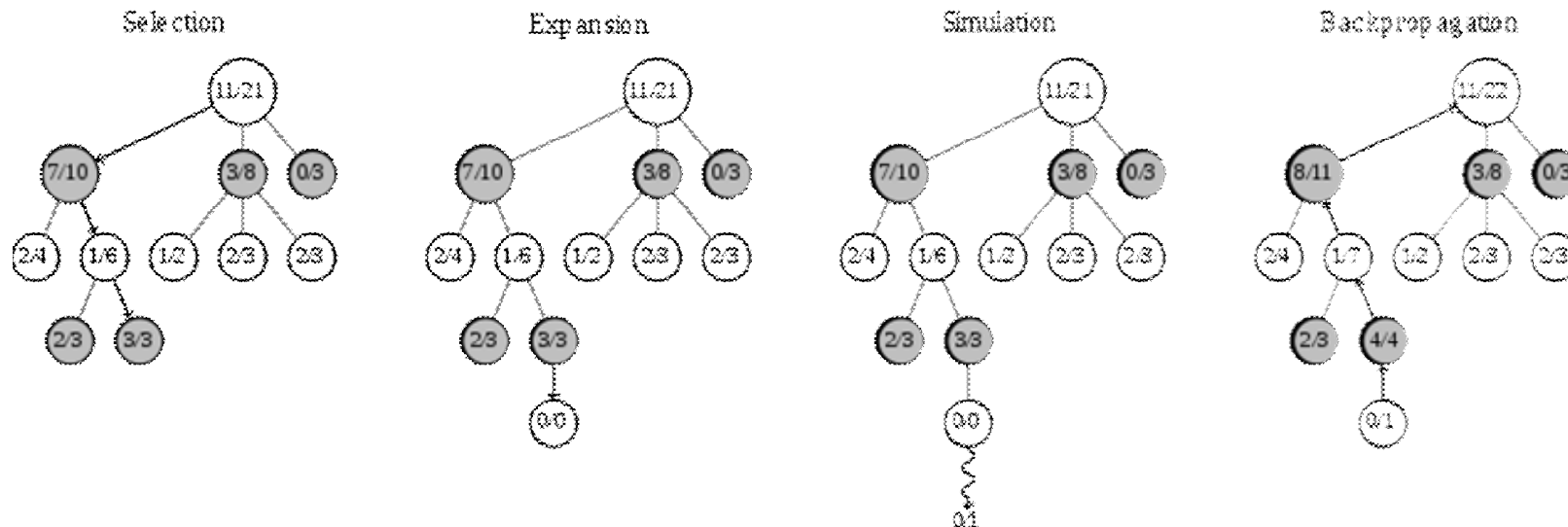


$$\Delta\theta = \frac{\alpha}{m} \sum_{k=1}^m \left( z^k - v_\theta(s^k) \right) \frac{\partial v_\theta(s^k)}{\partial \theta}$$

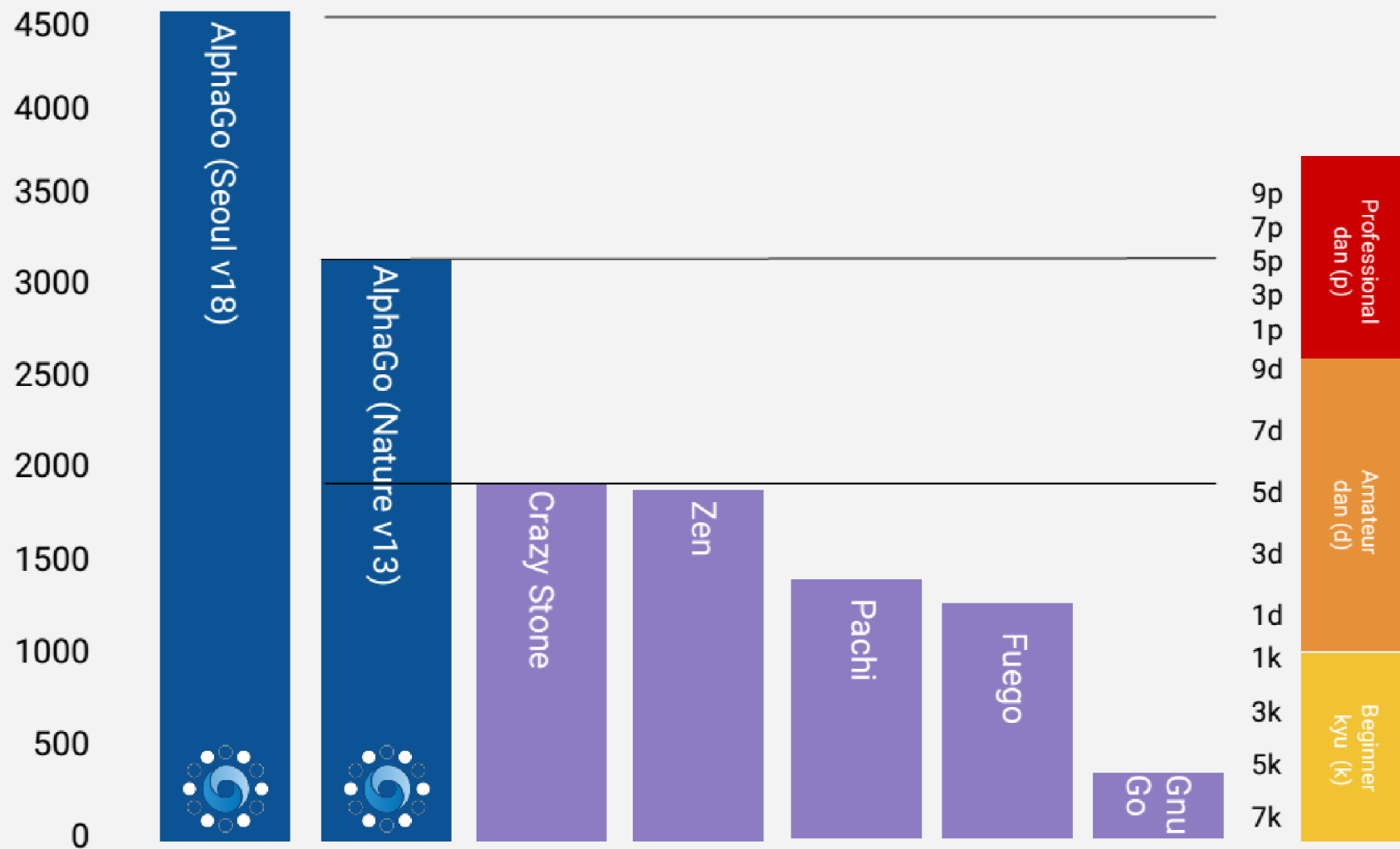


# Monte Carlo Tree Search

- The focus of Monte Carlo tree search is on the analysis of the most promising moves, expanding the search tree based on **random sampling** of the search space. The application of Monte Carlo tree search in games is based on many *playouts*. In each playout, the game is played out to the very end by selecting moves at random. The final game result of each playout is then used to weight the nodes in the game tree so that better nodes are more likely to be chosen in future playouts.



# Evaluating AlphaGo against computers

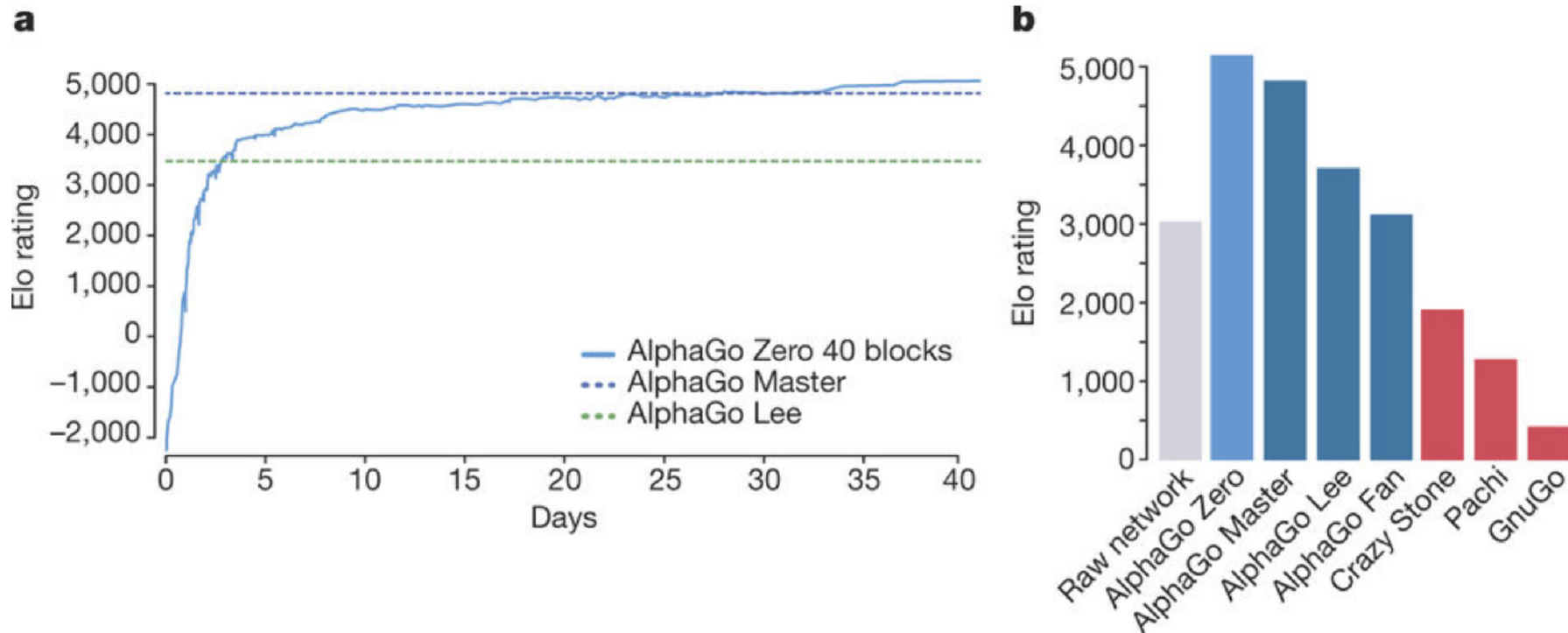


# AlphaGo Zero beat AlphaGo 100-0 in Oct 2017

- AlphaGo started with supervised learning based on human expert games, and then used reinforcement learning to tune the neural networks.
- AlphaGo Zero used no supervised learning, just reinforcement learning to tune the neural networks.
- AlphaGo Zero used a single neural network, rather than separate policy and value networks.

# AlphaGo Zero

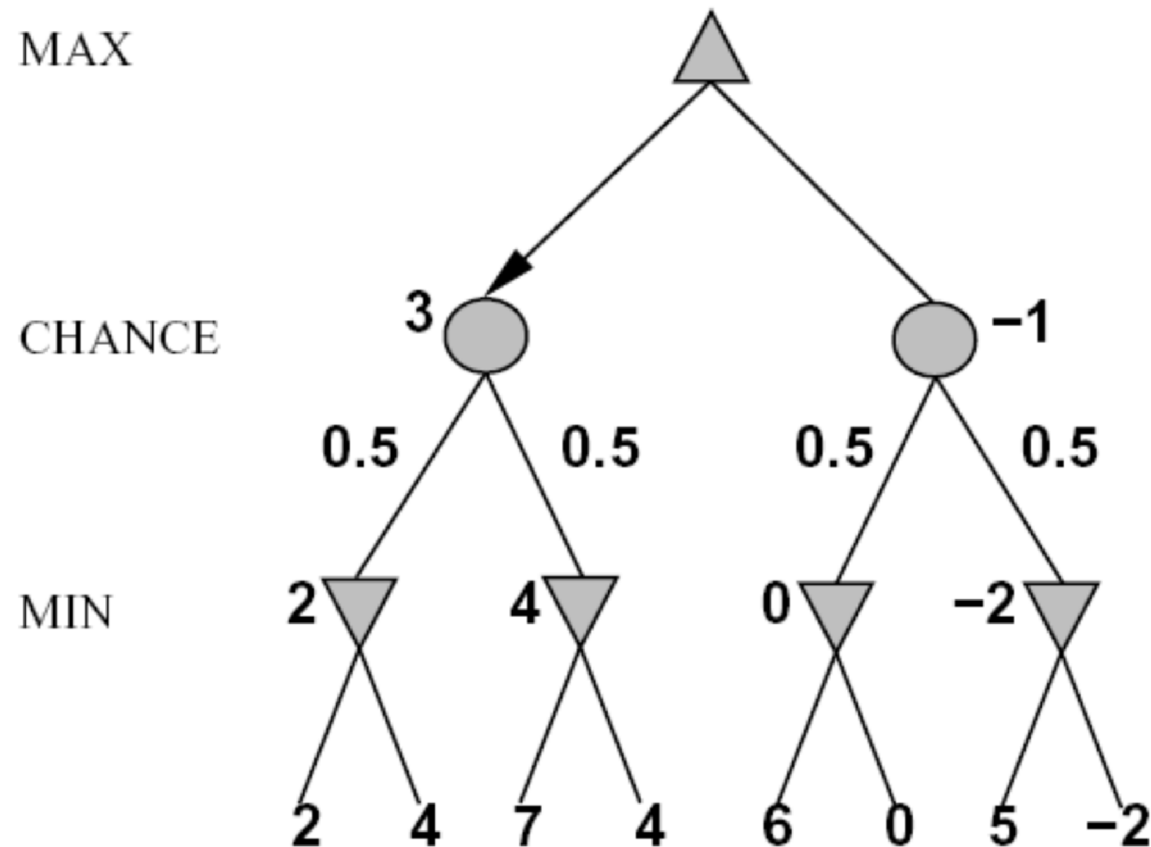
- Mastering the game of Go without human knowledge (Nature volume 550, pages 354–359)



## Nondeterministic games in general

In nondeterministic games, chance introduced by dice, card-shuffling

Simplified example with coin-flipping:





## Algorithm for nondeterministic games

EXPECTIMINIMAX gives perfect play

Just like MINIMAX, except we must also handle chance nodes:

...

**if** *state* is a MAX node **then**

**return** the highest EXPECTIMINIMAX-VALUE of SUCCESSORS(*state*)

**if** *state* is a MIN node **then**

**return** the lowest EXPECTIMINIMAX-VALUE of SUCCESSORS(*state*)

**if** *state* is a chance node **then**

**return** average of EXPECTIMINIMAX-VALUE of SUCCESSORS(*state*)

...

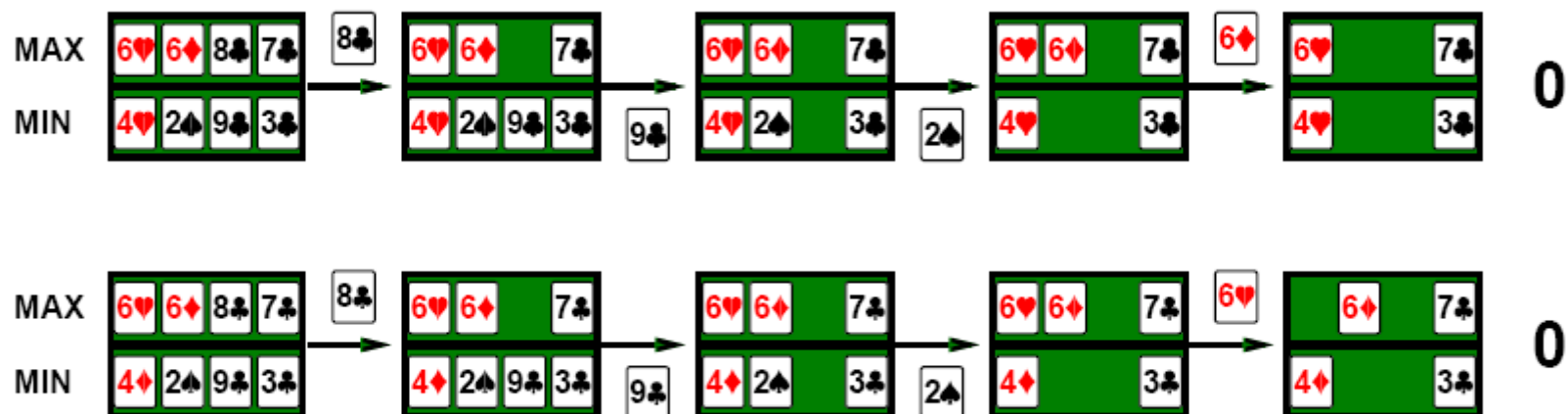
As depth increases, probability of reaching a given node shrinks  
 $\Rightarrow$  value of lookahead is diminished

$\alpha$ - $\beta$  pruning is much less effective

# 牌类游戏

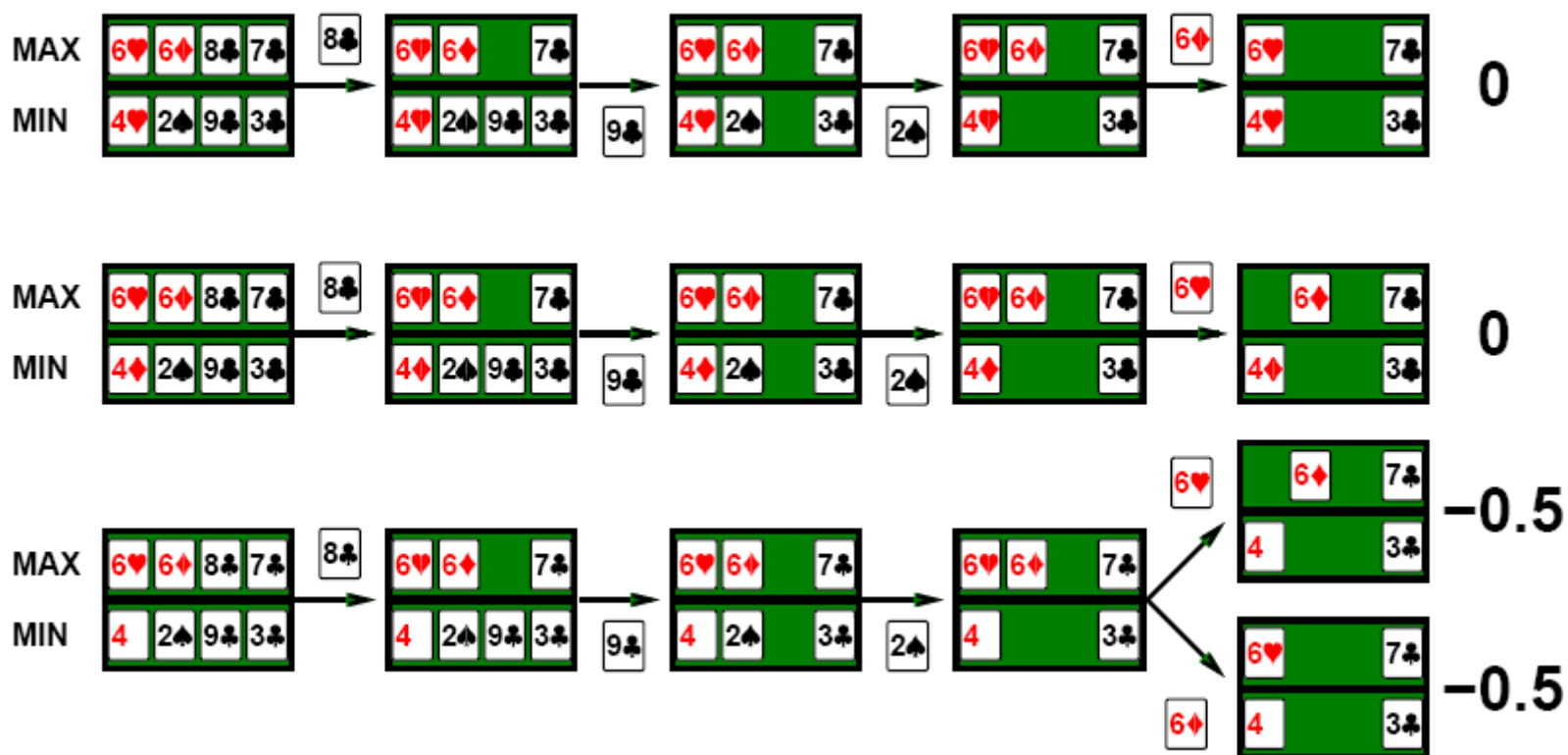


# 牌类游戏



# 牌类游戏

Four-card bridge/whist/hearts hand, MAX to play first



# 分析：打牌不同于扔骰子

\* Intuition that the value of an action is the average of its values in all actual states is **WRONG**

With partial observability, value of an action depends on the **information state** or **belief state** the agent is in

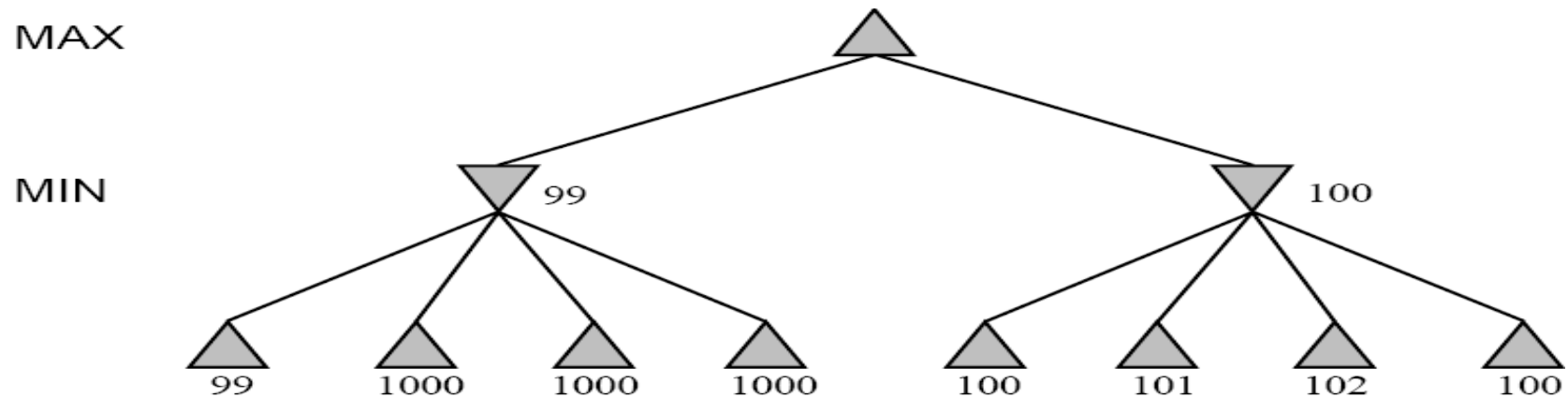
Can generate and search a tree of information states

Leads to rational behaviors such as

- ◇ Acting to obtain information
- ◇ Signalling to one's partner
- ◇ Acting randomly to minimize information disclosure

# Discussions

- Typical situations: evaluations are approximate



- Instead of straightforward *min* or max calculation, MINIMAX can return a probability distribution over possible values.

- Improvement: use Metareasoning
  - $\alpha$ – $\beta$  search (which is also a kind of simple metareasoning) reduces search space considerably, however, most searches are still of little value
  - To further improve efficiency, we should only select nodes of high utility for expansion
- Alternative to search: goal-directed reasoning
  - Humans often has a particular goal in mind in chess playing - for example, trapping the opponent's queen
  - How to combine search and goal-directed reasoning into a robust and efficient system?



# Summary

- Games are fun to work on!
- They illustrate several important points about AI
- - ◇ perfection is unattainable  $\Rightarrow$  must approximate
  - ◇ good idea to think about what to think about
  - ◇ uncertainty constrains the assignment of values to states
  - ◇ optimal decisions depend on information state, not real state

# 作业

- Prove the following assertion: for every game tree, the utility obtained by MAX using minimax decisions against a suboptimal MIN will be never be lower than the utility obtained playing against an optimal MIN. Can you come up with a game tree in which MAX can do still better using a suboptimal strategy against a suboptimal MIN?

# 论文选读

- Claude Shannon: Programming a computer for playing chess, 1950
- Matthew Lai, Giraffe: Using Deep Reinforcement Learning to Play Chess. Sept. 2015
- David Silver et al: Mastering the game of Go with deep neural networks and tree search, Nature 529, 484–489 (28 January 2016)

# 论文选读

- Mastering the game of Go without human knowledge“. Nature. 19 October 2017. 这篇论文存储采用强化学习算法，没有采用人类知识！(By playing games against itself, AlphaGo Zero surpassed the strength of [AlphaGo Lee](#) in three days by winning 100 games to 0, reached the level of [AlphaGo Master](#) in 21 days, and exceeded all the old versions in 40 days.)

# 课程设计

- 写一个象棋/围棋程序。
  - 建议：
  - 阅读本章有关下国际象棋/围棋的文献
  - 下载前人有关程序
  - 中国象棋比较简单。
- 写一个牌类程序，最好做成一个软件机器人，能在网上伪装成人用（如QQ中）。